

Introduzione

Il **BucketSort** nello specifico ordina dei *record*, ovvero strutture contenenti più tipi di dati. La sua tecnica si basa sui così detti **bucket**, ovvero celle all'interno di una **struttura ausiliaria** che ci aiutano a ordinare gli elementi della struttura originale.

Algorithm 1: BUCKETSORT(A, n, k)

```
Input: Array  $A$  di lunghezza  $n$ , numero  $k$  di bucket
// Dove  $n$  è la lunghezza di  $A$  e  $k$  è il numero di bucket
// Istanzio l'array ausiliario  $Y$  di dimensione  $k$ 
for  $i = 1$  to  $k$  do
     $Y[i] \leftarrow$  coda vuota

for  $i = 1$  to  $n$  do
    if  $A[i].chiave \in [1 \dots k]$  then
         $Y[A[i].chiave].enqueue(A[i])$ 

// Contatore di  $A$ 
 $j \leftarrow 1$ ;
for  $i = 1$  to  $k$  do
    while not  $Y[i].empty$  do
         $A[j] \leftarrow Y[i].dequeue()$ ;
         $j \leftarrow j + 1$ ;
```

L'algoritmo quindi popolerà la *struttura ausiliaria* Y con elementi aventi la stessa radice oer poi ricopiarli, in maniera ordinata, nella *struttura originale*. Inoltre è una tecnica **stabile** poichè se due elementi con la stessa chiave finiscono nello stesso bucket "usciranno" nello stesso ordine in cui sono entrati.

Costi computazionali

L'algoritmo dipende soltanto da due variabili, n e k , quindi il suo costo è semplicemente $O(n + k)$, ma se k è direttamente proporzionale a n allora il costo diventerà **lineare**: $O(n)$,

Allo stesso modo però se ci ritroviamo con un $k = n^2$ il costo si alzerà ancora di più.